FIRST AND FOLLOW

EX. NO. 5

Meghavarshini M (RA2211026050054)

AIM: To write a program to covert find the first and follow of a given production.

ALGORITHM:

1. Start.
2. Initialize FIRST and FOLLOW sets as empty for all non-terminals.
3. Compute FIRST Set:
    1. If the symbol is a terminal, add it to FIRST.
    2. If the symbol is a non-terminal, process its productions.
    3. For each production, process symbols from left to right.
    4. Recursively compute FIRST for each symbol in the production.
    5. If a symbol's FIRST contains 'e', continue to the next symbol.
    6. If all symbols in a production can derive 'e', add 'e' to FIRST.
4. Compute FOLLOW Set:
    1. Add '$' to FOLLOW of the start symbol.
    2. Iterate over productions to find occurrences of each non-terminal.
    3. If a symbol is followed by another, add its FIRST (except 'e') to FOLLOW.
    4. If a symbol is at the end or followed by symbols that can derive 'e', add FOLLOW of the left-hand side to FOLLOW.
5. Repeat until FIRST and FOLLOW sets remain unchanged.
6. Stop.

PROGRAM:

```
#include <iostream>

#include <map>

#include <set>

#include <vector>


using namespace std;
```

```
class Grammar {
public:
    map<char, vector<string>> productions;
    map<char, set<char>> firsts, follows;

    Grammar(map<char, vector<string>> prods) : productions(prods) {
        computeFirsts();
        computeFollows();
    }

    set<char> computeFirst(char symbol) {
        if (firsts.count(symbol)) return firsts[symbol];
        set<char> first;
        if (!isupper(symbol)) {
            first.insert(symbol);
        } else {
            for (auto &prod : productions[symbol]) {
                bool epsilonFound = true;
                for (char ch : prod) {
                    set<char> tempFirst = computeFirst(ch);
                    first.insert(tempFirst.begin(), tempFirst.end());
                    first.erase('e');
                    if (tempFirst.find('e') == tempFirst.end()) {
                        epsilonFound = false;
```

```
                break;
            }
        }
        if (epsilonFound) first.insert('e');
    }
}
return firsts[symbol] = first;
}

void computeFirsts() {
    for (auto &[nonTerminal, _] : productions) {
        computeFirst(nonTerminal);
    }
}

set<char> computeFollow(char symbol) {
    if (follows.count(symbol)) return follows[symbol];
    set<char> follow;
    if (symbol == productions.begin()->first) follow.insert('$');
    for (auto &[lhs, rhsList] : productions) {
        for (auto &rhs : rhsList) {
            for (size_t i = 0; i < rhs.size(); i++) {
                if (rhs[i] == symbol) {
                    bool epsilonFound = false;
```

```
                if (i + 1 < rhs.size()) {
                    set<char> tempFirst = computeFirst(rhs[i + 1]);
                    follow.insert(tempFirst.begin(), tempFirst.end());
                    follow.erase('e');
                    epsilonFound = tempFirst.find('e') != tempFirst.end();
                }
                if (i + 1 == rhs.size() || epsilonFound) {
                    if (lhs != symbol) {
                        set<char> tempFollow = computeFollow(lhs);
                        follow.insert(tempFollow.begin(), tempFollow.end());
                    }
                }
            }
        }
    }
    return follows[symbol] = follow;
}

void computeFollows() {
    for (auto &[nonTerminal, _] : productions) {
        computeFollow(nonTerminal);
    }
}
```

```
    void display() {
        cout << "FIRST sets:\n";
        for (auto &[nonTerminal, firstSet] : firsts) {
            cout << "FIRST(" << nonTerminal << ") = { ";
            for (char ch : firstSet) cout << ch << " ";
            cout << "}\n";
        }
        cout << "\nFOLLOW sets:\n";
        for (auto &[nonTerminal, followSet] : follows) {
            cout << "FOLLOW(" << nonTerminal << ") = { ";
            for (char ch : followSet) cout << ch << " ";
            cout << "}\n";
        }
    }
};

int main() {
    map<char, vector<string>> grammar_rules = {
        {'E', {"TA"}},
        {'A', {"+TA", "e"}},
        {'T', {"FB"}},
        {'B', {"*FB", "e"}},
        {'F', {"(E)", "id"}}
```

```
    };
    Grammar grammar(grammar_rules);
    grammar.display();
    return 0;
}
```

INPUT(STRING):

```
{
        {'E', {"TA"}},
        {'A', {"+TA", "e"}},
        {'T', {"FB"}},
        {'B', {"*FB", "e"}},
        {'F', {"(E)", "id"}}
    }
```

OUTPUT:

```
FIRST sets:
FIRST(() = { ( }
FIRST()) = { ) }
FIRST(*) = { * }
FIRST(+) = { + }
FIRST(A) = { + e }
FIRST(B) = { * e }
FIRST(E) = { ( i }
FIRST(F) = { ( i }
FIRST(T) = { ( i }
FIRST(e) = { e }
FIRST(i) = { i }

FOLLOW sets:
FOLLOW(A) = { $ ) }
FOLLOW(B) = { $ ) + }
FOLLOW(E) = { ) }
FOLLOW(F) = { $ ) * + }
FOLLOW(T) = { $ ) + }
```